

DoctorDirect

Complete Project Documentation

Offline-First Doctor Appointment Booking, Tele-Consultation & AI-Assisted Consultation Records

Field	Value
Project Type	Application Development
Team	Chinthaginjala Madhav Sai Kiran & Charan Tej Guniganti
Year & Section	Sem 3 A
Duration	10 Weeks
Core Platform	React Native + Expo + TypeScript
Backend	Node.js + Express.js + PostgreSQL
Offline Storage	SQLite
Real-Time Consultation	WebRTC
AI / Speech	Groq API — Speech-to-Text + LLM summary

A unified healthcare mobile application that connects patients and doctors through appointment booking, online consultation, offline access to important records, AI-assisted consultation summaries, and doctor-approved digital prescriptions.

Prepared as an OJT project documentation package • Version 1.0 • Draft

Table of Contents

#	Chapter
1	Project Overview
2	Business Requirements (BRD)
3	Product Requirements (PRD)
4	UX Requirements
5	Technical Requirements (TRD)
6	High-Level Design (HLD)
7	Database Design
8	API Specification
9	Low-Level Design (LLD)
10	Frontend Architecture
11	Security Design
12	Testing Strategy
13	CI/CD Pipeline
14	Observability
15	Deployment Architecture
16	Cost Analysis
17	Project Roadmap
18	Team Responsibilities
19	GitHub Repository Structure
20	README
21	Architecture Decision Records (ADRs)

1. Project Overview — DoctorDirect

DoctorDirect is an OJT mobile application for patients and medical clinics/doctors. It brings doctor discovery, appointment scheduling, online consultation and prescription access into one mobile workflow, with offline access to previously synchronized important information.

1.1 Project Identity

Field	Value
Project Name	DoctorDirect
Domain	Healthcare / Mobile Application
Track	Application Development
Team Size	2 Students
Duration	10 Weeks
Level	Intermediate
Primary Users	Patients and Doctors / Medical Clinics

1.2 Problem Summary

- Patients may need separate methods to find doctors, check availability, book appointments, attend consultations and access prescriptions.
- Poor or unavailable connectivity can make previously available healthcare information difficult to access.
- After an online consultation, users benefit from a clear record of what was discussed, recommendations and prescribed medicines.
- Doctors need a controlled workflow to review and approve any AI-assisted consultation summary before it becomes a final record.

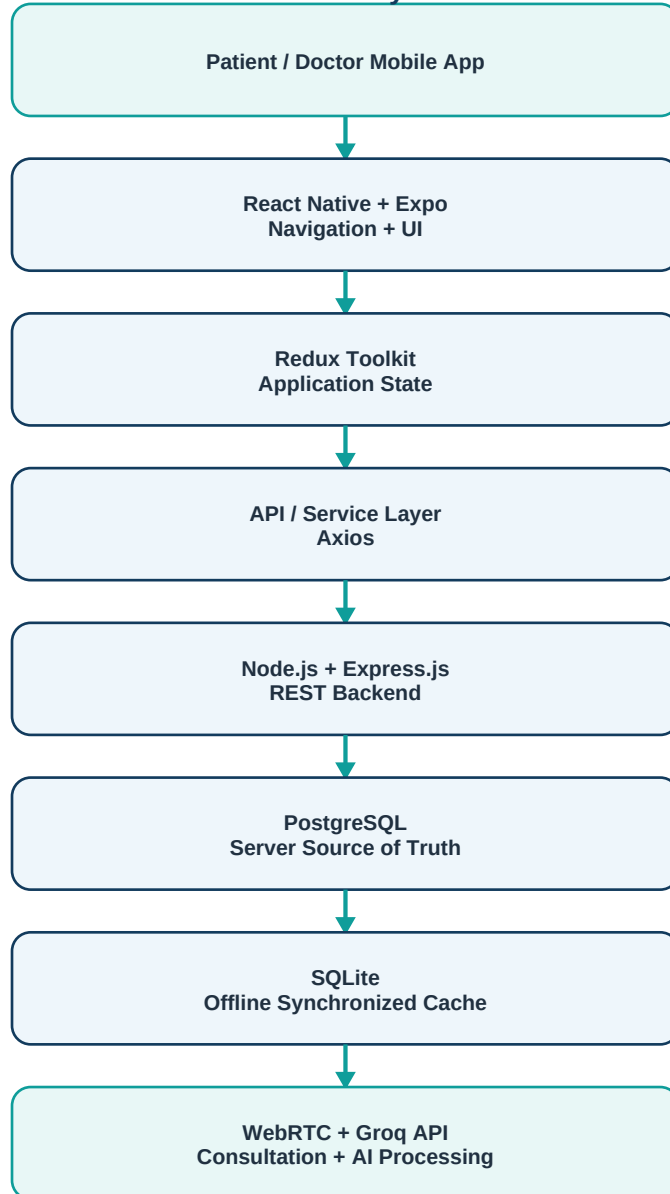
1.3 Proposed Solution

DoctorDirect combines patient and doctor workflows in one React Native application with role-based access. The mobile app communicates with a backend API and PostgreSQL database while maintaining selected synchronized information in SQLite for offline access.

Core capabilities include doctor search by specialization, appointment and slot booking, doctor-side appointment management, WebRTC consultation, temporary consultation-audio processing, speech-to-text, AI-assisted summary generation, doctor review/approval, prescription creation and offline access.

1.4 System Architecture Overview

DoctorDirect — Vertical System Architecture



2. Business Requirements Document (BRD)

2.1 Executive Summary

The product is designed to reduce friction across the patient journey from finding a doctor to receiving a consultation record and prescription. The business objective for the OJT project is a demonstrable end-to-end workflow with reliable role separation, appointment handling, consultation, offline access and testing.

2.2 Target Users

Persona	Goals	Pain Points
Patient	Find appropriate doctor, book a slot, attend consultation, access records	Multiple systems; connectivity interruptions
Doctor	Manage schedule, attend consultations, review summaries, issue prescriptions	Appointment coordination; manual record preparation
Clinic / Mentor Demo Role	See controlled workflows and data handling	Need clear role-based access and reliable demo flow

2.3 Business Objectives

- Provide one mobile workflow for doctor discovery, appointment booking, consultation and prescription access.
- Keep previously synchronized appointments and prescriptions available during network loss.
- Use AI only as an assistant for drafting consultation summaries; doctor review remains part of the workflow.
- Demonstrate secure role-based access, API integration, offline synchronization and automated testing.

2.4 Functional Business Requirements

ID	Requirement	Priority	Acceptance Direction
BR-001	Patient & doctor authentication with role-based dashboards	Must Have	Correct role receives correct dashboard and permissions
BR-002	Doctor search and profile by specialization	Must Have	Patient can browse and inspect doctor availability
BR-003	Appointment and slot booking	Must Have	Only valid available slots can be booked
BR-004	Doctor appointment management	Must Have	Doctor sees schedule and patient appointment details
BR-005	Rescheduling and cancellation	Must Have	Allowed changes update appointment state correctly
BR-006	WebRTC video consultation	Must Have	Patient and doctor can join the same scheduled session
BR-007	Offline access + synchronization	Must Have	Previously synced important records remain readable offline
BR-008	Prescription management	Must Have	Doctor creates/approves prescription; patient can view it
BR-009	AI-assisted consultation summary	Must Have	Transcript → draft summary → doctor review → final record
BR-010	Push notifications	Must Have	Booking/reminder/consultation events generate notifications

2.5 Non-Functional Requirements

Category	Requirement / Target
Performance	Smooth navigation and responsive UI under normal use
Reliability	No critical failure in the demonstrated end-to-end workflow
Offline	Previously synchronized important data remains accessible without internet
Security	Authenticated role-based access and protected healthcare-related data
Testing	80%+ coverage for critical application logic
Compatibility	Android/iOS build through Expo/EAS as feasible for the OJT scope

3. Product Requirements Document (PRD)

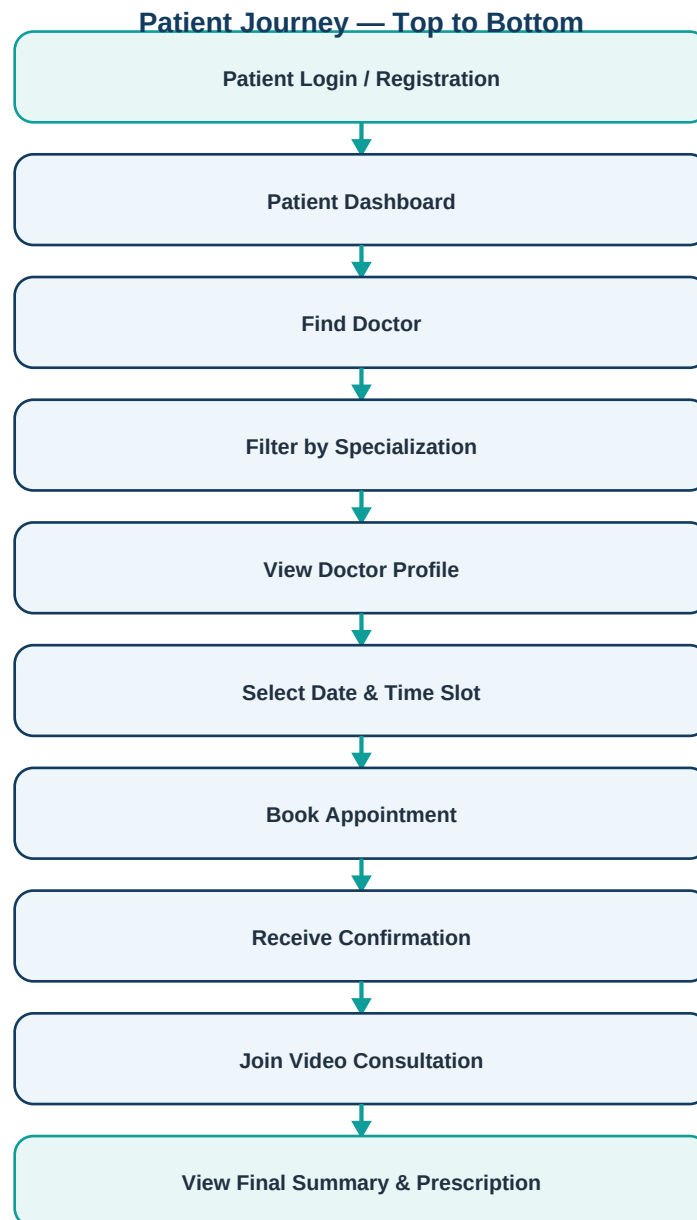
3.1 Product Vision

Provide a single mobile experience in which patients can find doctors, book appointments, attend online consultations and access doctor-approved records, while doctors retain control over scheduling, consultation summaries and prescriptions.

3.2 Feature Prioritization

Priority	Features
Must Have	Authentication; role-based dashboards; doctor search; specialization; slots; booking; rescheduling/cancellation; doctor appointment management; WebRTC; offline access; API sync; prescriptions; AI-assisted summary; notifications
Should Have	Dark mode; enhanced appointment history/search; doctor availability controls; profile/settings
Could Have	More advanced offline conflict handling; expanded analytics; deployment automation
Won't Have for MVP	Permanent consultation-video storage; phone-call fallback; Google Calendar integration; claims of legally valid digital signatures without an appropriate signing mechanism

3.3 Patient Journey



3.4 Doctor Journey



3.5 Core User Stories

- As a patient, I want to search doctors by specialization so that I can identify relevant doctors.
- As a patient, I want to see available slots and book one so that my appointment is confirmed.
- As a doctor, I want to view upcoming appointments and patient details so that I can manage my schedule.
- As a patient or doctor, I want to join a scheduled video consultation so that the consultation can occur remotely.
- As a doctor, I want to review an AI-generated consultation summary so that I can correct or approve it before it becomes final.
- As a patient, I want to view my final consultation record and prescription after the doctor approves them.
- As a user, I want previously synchronized important records to remain readable when connectivity is unavailable.

4. UX Requirements

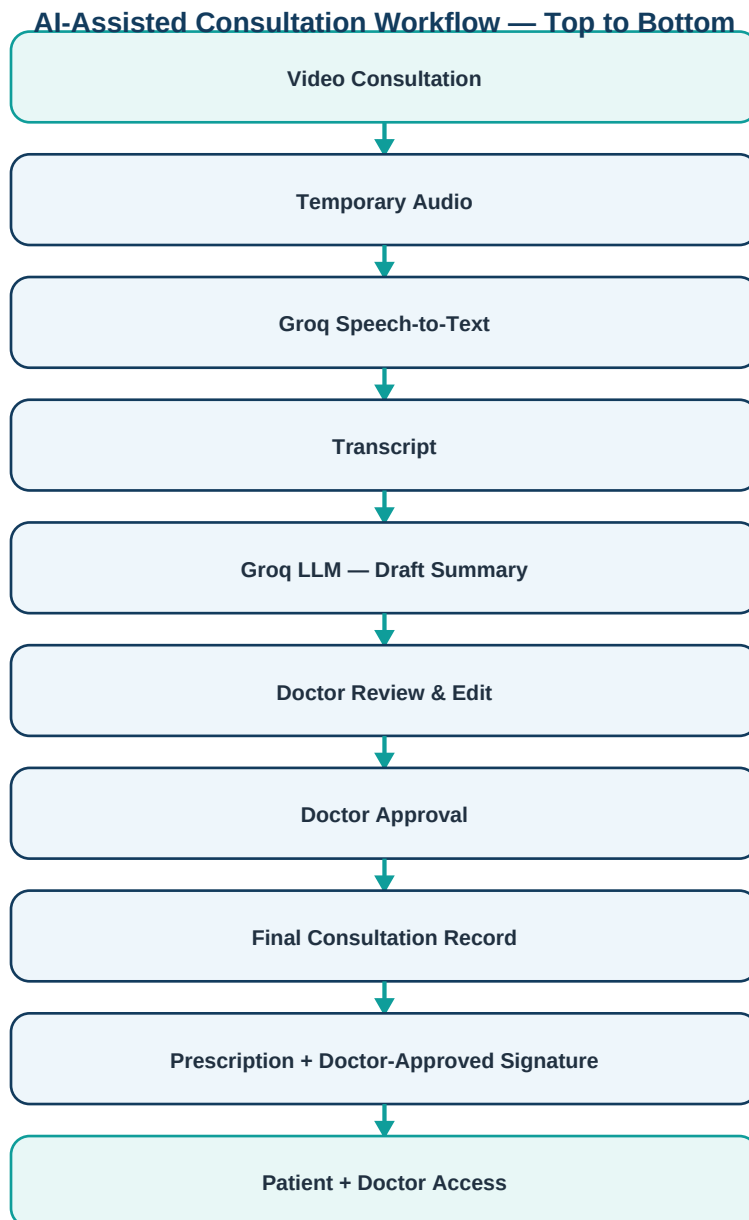
4.1 Information Architecture

DoctorDirect App ■■■ Authentication ■■■ Patient Login / Registration ■■■ Doctor Login / Registration ■■■ Patient Dashboard ■■■ Find Doctors ■■■ Specialization Filter ■■■ Doctor Profile ■■■ Slots / Booking ■■■ Appointments ■■■ Consultation ■■■ Prescriptions / Records ■■■ Doctor Dashboard ■■■ Today / Upcoming Appointments ■■■ Patient Details ■■■ Schedule / Availability ■■■ Consultation ■■■ AI Summary Review ■■■ Prescription Creation ■■■ Shared ■■■ Notifications ■■■ Profile / Settings ■■■ Offline State / Sync Status

4.2 Screen Requirements

Screen	Primary Components	Key States
Patient Dashboard	Upcoming appointment, doctor card, consultation status	Loading, empty, offline
Doctor Search	Search, specialization filter, doctor cards	Loading, no results, offline cache
Doctor Profile	Specialization, experience, fee/availability, slots	Available, unavailable
Slot Booking	Date selector, time slots, confirmation	Available, conflict, network error
Doctor Dashboard	Appointments, patient information, schedule	Today, upcoming, empty
Video Consultation	Local/remote video, controls, connection state	Connecting, connected, disconnected
AI Summary Review	Transcript/draft summary, edit, approve	Generating, editable, approved
Prescription	Medicines, dosage, duration, instructions, doctor approval/signature	Draft, approved, viewed
History / Records	Appointments, consultation summaries, prescriptions	Offline-readable if synchronized

4.3 Consultation UX



The temporary-audio approach is intended to avoid permanent storage of raw consultation video in the MVP. The AI output is a draft; the doctor must review and approve the summary before it becomes the final consultation record.

5. Technical Requirements Document (TRD)

5.1 Proposed Technology Stack

Layer	Technology	Purpose
Mobile	React Native + Expo	Cross-platform iOS/Android application
Language	TypeScript	Typed application development
Navigation	React Navigation	Stack/tab navigation and role-specific flows
State	Redux Toolkit	Application and role/appointment state
Local DB	SQLite / Expo SQLite	Offline cache and synchronized records
Backend	Node.js + Express.js	REST API and business logic
Database	PostgreSQL	Authoritative server-side data
Consultation	WebRTC	Real-time video communication
HTTP	Axios	REST API communication
Speech / AI	Groq API	Speech-to-text and AI-assisted draft summary

5.2 Technical Constraints

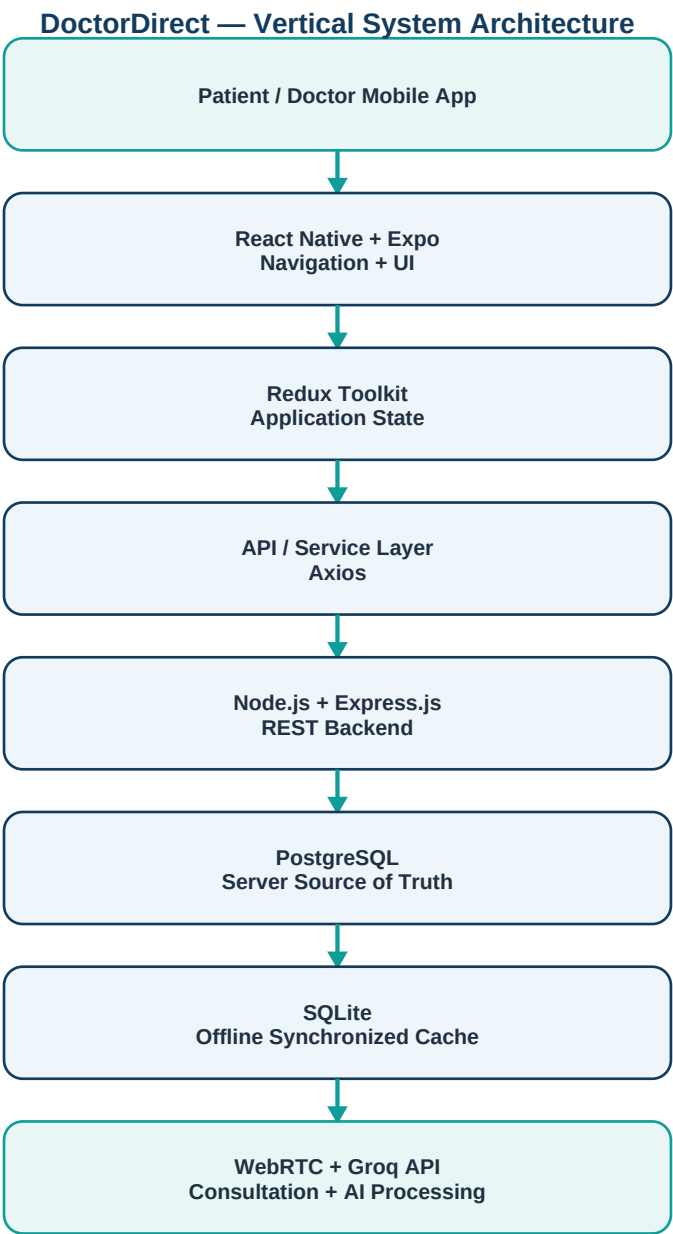
- Appointment booking requires server authority because two users cannot safely claim the same slot offline.
- Offline mode is intended for previously synchronized important information rather than unrestricted offline healthcare operations.
- WebRTC consultation requires network connectivity.
- Raw consultation video is not intended to be permanently stored in the MVP.
- AI-generated summaries must be treated as drafts and reviewed by the doctor.
- Any digital signature implementation should be described as a doctor-approved signature mechanism unless a legally compliant electronic-signature system is actually implemented.

5.3 Technical Requirements

ID	Requirement	Target
TR-01	Role-based authentication	Patient/doctor access separation
TR-02	API communication	Authenticated REST requests with validation and error handling
TR-03	SQLite synchronization	Cache selected records; sync when connectivity returns
TR-04	Appointment validation	Server validates slot availability before confirmation
TR-05	WebRTC	Connection lifecycle, permissions and disconnection handling
TR-06	STT	Send temporary consultation audio to selected STT API
TR-07	AI summary	Generate structured draft from transcript
TR-08	Doctor approval	No AI draft becomes final record without review
TR-09	Testing	Jest for critical application logic; integration testing for core workflows

6. High-Level Design (HLD)

6.1 End-to-End System Flow

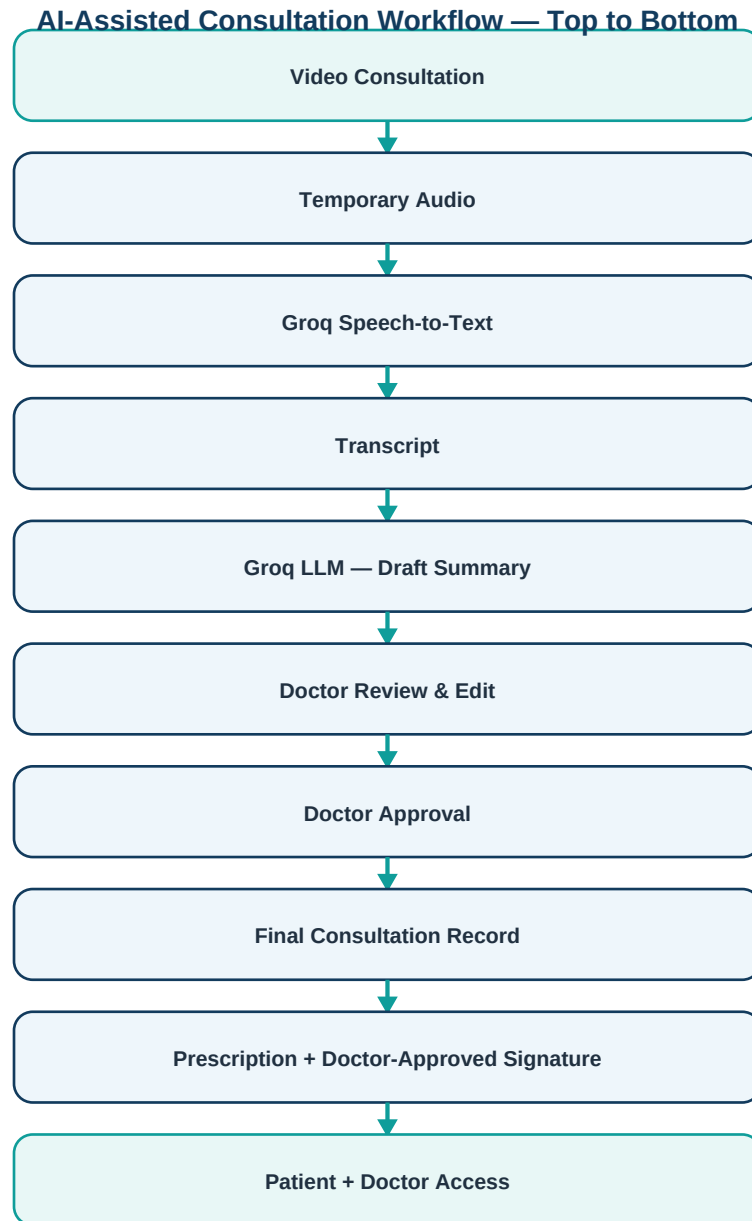


6.2 Major Components

Component	Responsibility
React Native UI	Patient and doctor screens, navigation, forms and consultation UI
Redux Toolkit	Authentication/session state, role state, appointment state and UI state
SQLite	Offline cache for synchronized appointments, prescriptions, summaries and other selected data
REST API	Authentication, doctor search, slots, appointments, consultations, prescriptions and sync endpoints
PostgreSQL	Authoritative persistent data store
WebRTC	Real-time consultation media path

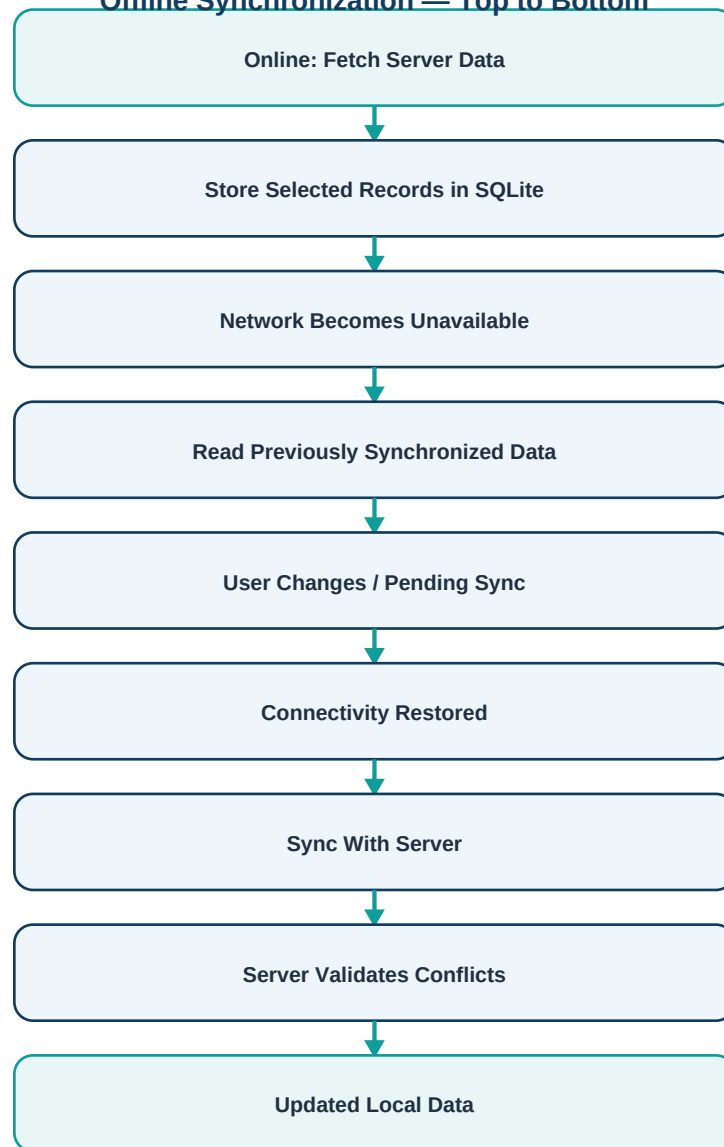
Component	Responsibility
Groq STT	Convert temporary consultation audio into transcript
Groq LLM	Generate structured draft consultation summary from transcript

6.3 Consultation Processing Flow



6.4 Offline Synchronization Flow

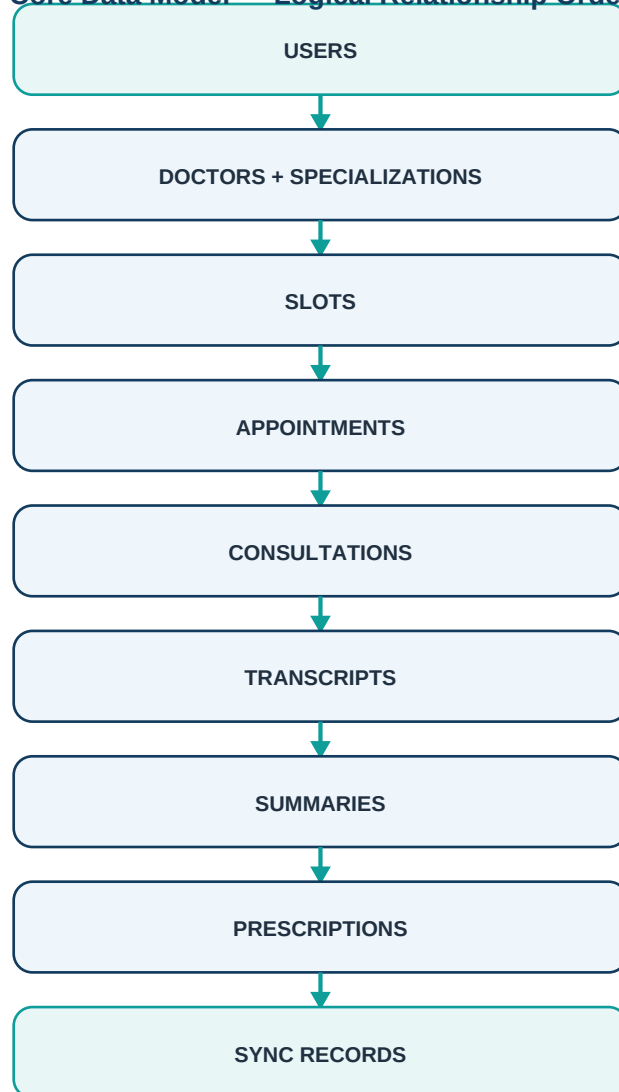
Offline Synchronization — Top to Bottom



7. Database Design

7.1 Core Data Model

Core Data Model — Logical Relationship Order



7.2 Core Entities

Entity	Important Fields / Purpose
users	id, name, phone/email, password hash or auth reference, role
doctors	user_id, specialization, experience, fee, profile, availability metadata
specializations	id, name/category used for doctor filtering
slots	doctor_id, date, start_time, end_time, availability/status
appointments	patient_id, doctor_id, slot_id, status, booked_at, reschedule/cancel data
consultations	appointment_id, room/session id, started_at, ended_at, status
transcripts	consultation_id, transcript text, processing metadata
summaries	consultation_id, draft text, reviewed text, approval status, timestamps

Entity	Important Fields / Purpose
prescriptions	appointment/consultation id, medicines, dosage, duration, instructions, approval/signature metadata
sync_records	local entity id, server id, sync state, updated_at, conflict metadata

7.3 Data Ownership Principle

PostgreSQL is the server-side source of truth. SQLite holds synchronized local copies needed for offline access. For appointment conflicts, the server must decide whether a booking is valid.

8. Repository / API Specification

8.1 Proposed REST Endpoints

Method	Endpoint	Purpose
POST	/auth/register	Create patient/doctor account
POST	/auth/login	Authenticate user and return session/token
GET	/doctors	List/filter doctors
GET	/doctors/:id	Doctor profile and availability
GET	/doctors/:id/slots	Available slots
POST	/appointments	Create appointment after server validation
PATCH	/appointments/:id	Reschedule/cancel/update appointment
GET	/appointments	Patient/doctor appointment list
POST	/consultations/:id/session	Create/prepare consultation session
POST	/consultations/:id/transcript	Submit/process consultation transcript metadata
POST	/consultations/:id/summary	Create draft summary / save reviewed summary
POST	/prescriptions	Create doctor prescription
GET	/prescriptions	List patient prescriptions
POST	/sync	Synchronize eligible local changes/data

8.2 API Processing Pattern

Mobile UI → Redux/Service Layer → Axios → REST API → Authentication/Validation → Business Logic → PostgreSQL ■ response → Redux → SQLite cache → UI

8.3 Error Handling

- 401/403: authentication or authorization failure.
- 404: requested doctor, appointment or record does not exist.
- 409: appointment slot conflict; client refreshes server availability.
- 422: validation error; client highlights invalid fields.
- 5xx/network timeout: preserve local readable data and retry eligible synchronization operations.

9. Low-Level Design (LLD)

9.1 Appointment Booking Logic

1. Patient selects doctor 2. App requests current slot availability 3. Patient selects slot 4. Client submits appointment request 5. Server re-validates doctor + slot + patient eligibility 6. If available → create appointment 7. Mark slot unavailable / reserved 8. Return confirmation 9. Store synchronized appointment locally 10. Trigger notification

9.2 AI Summary Logic

Consultation ends → temporary audio available → send audio to Speech-to-Text service → receive transcript → validate transcript payload → send transcript to LLM with structured summary prompt → receive draft → store as draft → doctor edits/reviews → doctor approves → persist final consultation record → notify patient → synchronize record to SQLite

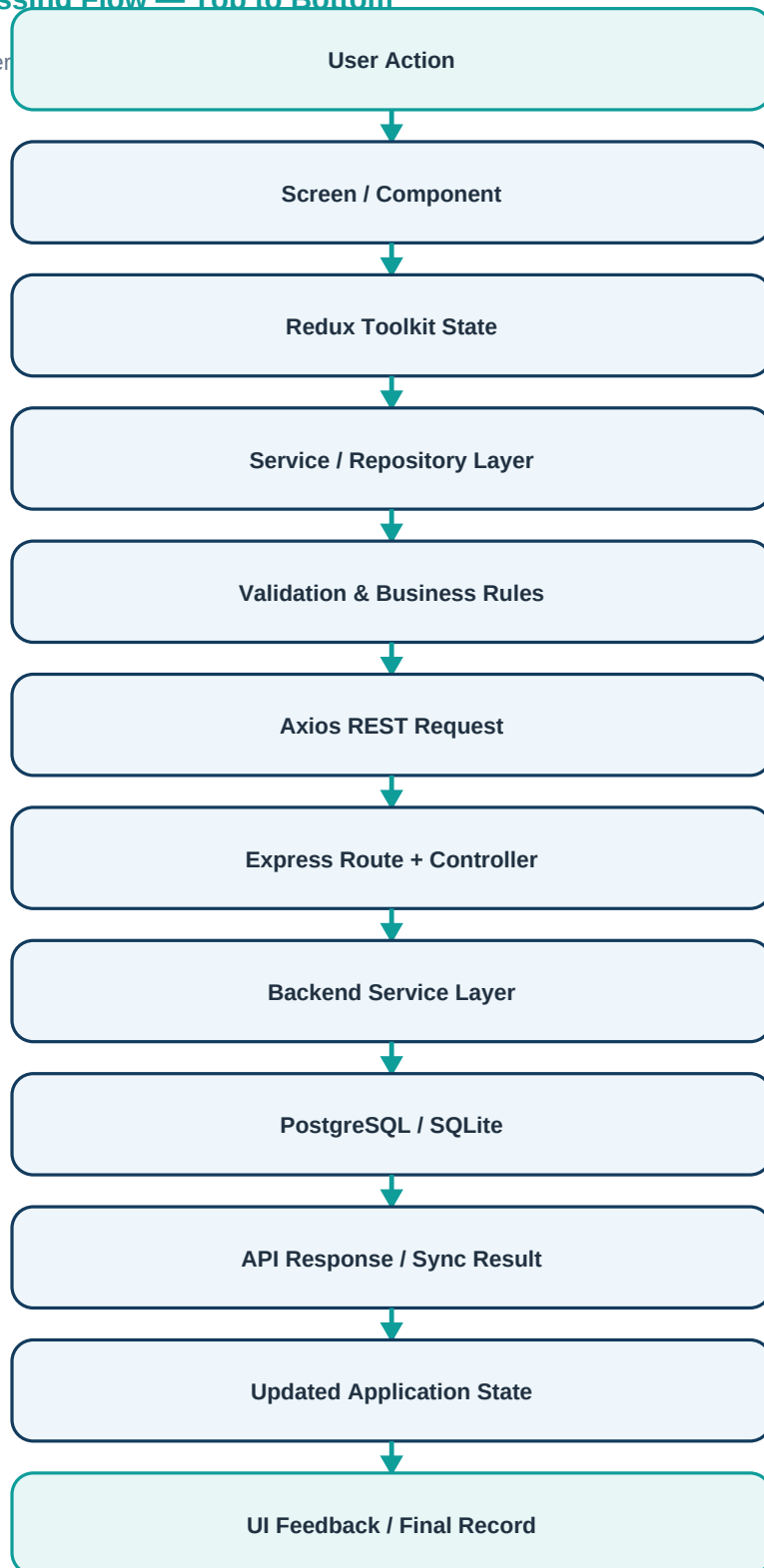
9.3 Validation Rules

Area	Rules
Authentication	Required credentials; role must match protected route
Booking	Doctor and slot required; slot must be available; duplicate active booking prevented
Rescheduling	New slot must be available and server-confirmed
Prescription	Medicine, dosage, duration/instructions validated before approval
AI Summary	Draft cannot be published as final without doctor approval
Offline	Read operations allowed for synchronized records; server-required booking actions are not treated as confirmed offline

9. Low-Level Design (LLD)

9.1 Request / Processing Flow — Top to Bottom

Detailed path from user inter



How this flow is applied

- Appointment booking: validate the selected slot on the server before confirming it.
- Consultation: manage WebRTC session state, then process temporary audio through STT and AI summary.
- Prescription: validate doctor-entered data, save approval state, then expose the final record to the patient.
- Offline sync: SQLite provides local access while the server remains authoritative for conflicts.

10. Frontend Architecture

10.1 Recommended Structure

```
src/ ■■■ navigation/ ■■■ screens/ ■ ■■■ auth/ ■ ■■■ patient/ ■ ■■■ doctor/ ■ ■■■ consultation/ ■■■  
components/ ■■■ store/ ■■■ services/ ■ ■■■ api/ ■ ■■■ auth/ ■ ■■■ consultation/ ■ ■■■ sync/ ■■■  
database/ ■ ■■■ migrations/ ■ ■■■ repositories/ ■ ■■■ models/ ■■■ hooks/ ■■■ utils/ ■■■ types/
```

10.2 State Responsibilities

State Area	Examples
Auth	Current user, role, session/token, login state
Patient	Doctors, selected specialization, appointments, prescriptions
Doctor	Schedule, appointments, patient details, review state
Consultation	Room/session status, connection status, transcript/summary workflow
Sync	Connectivity state, pending sync items, last sync time
UI	Theme, loading/error states, navigation state

10.3 Reusable Components

- DoctorCard
- SlotPicker
- AppointmentCard
- VideoControls
- SummaryReviewEditor
- PrescriptionForm
- OfflineStatusIndicator
- SyncStatusCard

11. Security Design

11.1 Security Principles

- Least-privilege role-based access: patient and doctor routes are separated.
- Sensitive API operations require authenticated requests and server-side authorization.
- Passwords must never be stored in plaintext; use an appropriate authentication mechanism and password hashing if credentials are server-managed.
- Use HTTPS/TLS for network communication.
- Do not place API secrets directly in the mobile application.
- Keep temporary consultation audio and transcripts controlled and delete temporary media when processing is complete where appropriate.
- AI output is untrusted draft content until reviewed by the doctor.
- Maintain audit-relevant timestamps and approval state for consultation summaries and prescriptions.

11.2 Healthcare Data Considerations

The OJT project should demonstrate privacy-aware engineering, but it should not be described as legally compliant or production-ready healthcare software unless the necessary regulatory, privacy, consent, security, audit and signature requirements have actually been implemented and reviewed.

11.3 Threat / Mitigation Matrix

Risk	Mitigation
Unauthorized role access	Server-side role authorization + protected navigation
Token leakage	Secure token storage; never hard-code secrets
Appointment double booking	Server-side availability validation + conflict response
AI hallucination	Transcript-grounded prompt + mandatory doctor review
Transcript exposure	Minimal retention, access control, encrypted transport
Offline stale data	Display synchronization timestamp; server authority on conflicts
WebRTC session abuse	Authenticated session creation and short-lived room/session credentials

12. Testing Strategy

Test Type	Scope	Example
Unit	Business logic / utilities	Appointment validation, reducers, sync rules
Integration	App + API + DB boundaries	Login, doctor list, booking
Role-Based	Access control	Patient cannot access doctor-only operations
Offline	Read/cache/sync behavior	View synchronized prescription without internet
Synchronization	Reconnect/conflict behavior	Server reconciliation after network returns
WebRTC	Connection lifecycle	Join, disconnect, reconnect
STT	Transcript processing	Audio → transcript response handling
AI Summary	Fidelity and structure	Draft reflects transcript; no invented clinical facts
Prescription	Doctor approval workflow	Draft → review → final patient view
Performance	Navigation/render/network	Responsive core screens
Build	Android/iOS	Internal build and final demo

12.1 End-to-End Test

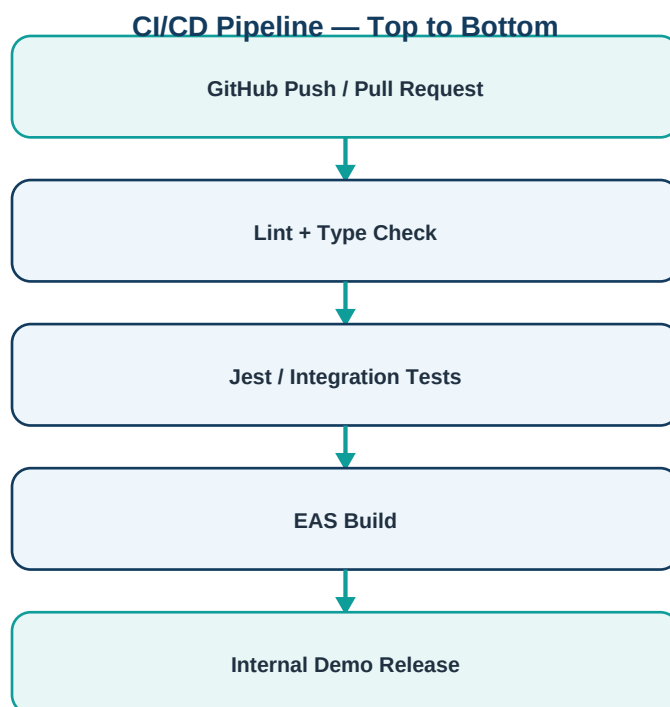
Login → doctor search → specialization filter → slot selection → booking → doctor confirmation → video consultation → temporary audio → transcription → AI draft summary → doctor review/approval → prescription → patient record → offline read → reconnect and synchronize.

12.2 Quality Target

Target: 80%+ automated coverage for critical application logic, plus manual/integration verification of WebRTC, synchronization and end-to-end workflows.

13. CI/CD Pipeline

13.1 Proposed Pipeline



13.2 Pipeline Stages

- Pull request: formatting/lint/type checks.
- Automated tests: Jest and integration tests available in the repository.
- Build: EAS development/internal build for device validation.
- Release candidate: integrated test run and mentor review.
- Final: production-style build for OJT demonstration, subject to project scope and available accounts/services.

13.3 Git Strategy

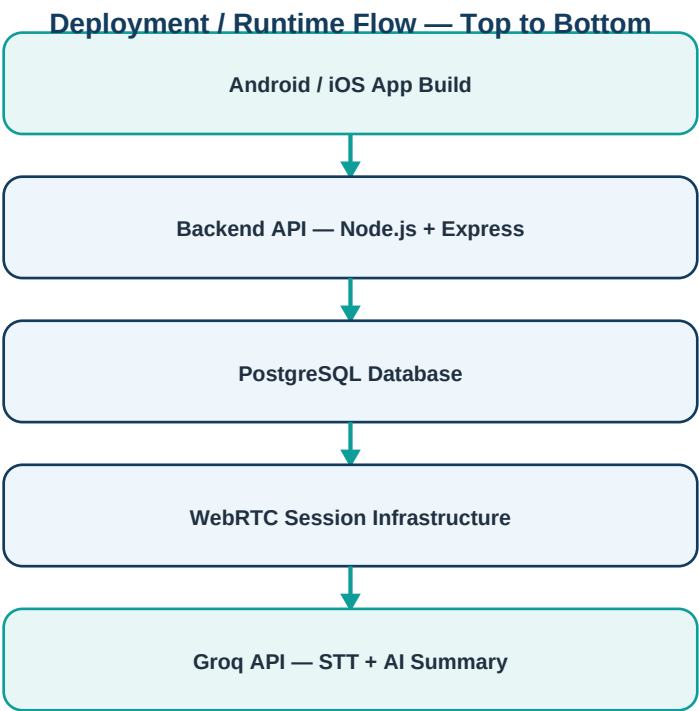
Recommended branches: main (stable), develop (integration), feature/* (individual work). Pull requests should include test evidence for core changes.

14. Observability

Signal	Purpose	Examples
Application errors	Identify crashes/runtime failures	JS error boundary, structured logs
API errors	Trace failed backend operations	HTTP status, request correlation ID
Sync state	Understand offline behavior	last sync time, pending count, failed sync count
Consultation state	Debug real-time sessions	connecting/connected/disconnected states
AI processing	Track pipeline failures	STT success, summary generation status, approval status
Performance	Detect slow screens	navigation timing, API latency, DB query timing

Observability should avoid logging raw medical content, full transcripts or prescription details unnecessarily. Logs should favor IDs, statuses, timings and error categories.

15. Deployment Architecture



15.1 Environments

Environment	Purpose
Development	Local React Native/Expo development and mock/test data
Integration	Mobile build connected to development backend/database
Demo	Stable backend and mobile build used for mentor/final presentation
Production	Future scope; would require additional healthcare-grade security, privacy, monitoring and compliance work

15.2 Build

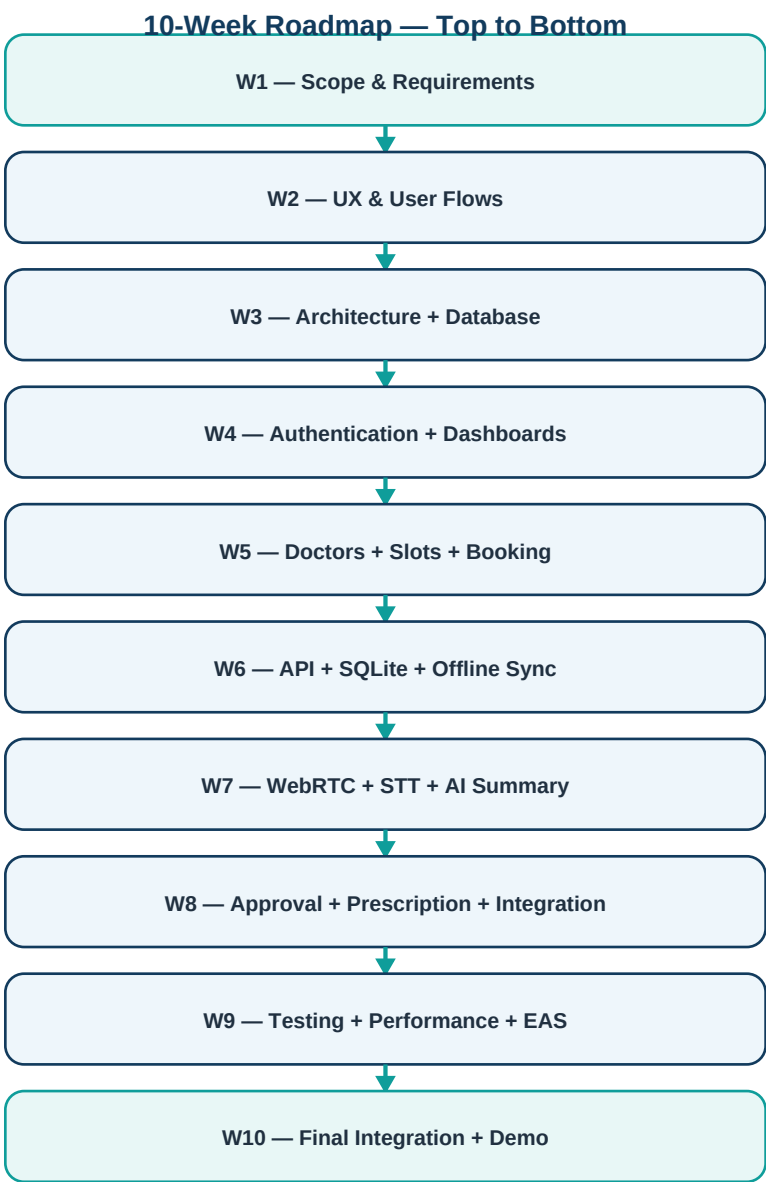
Use Expo/EAS for Android and iOS builds. A physical device is recommended for realistic WebRTC, camera/microphone permissions and network-condition testing.

16. Cost Analysis

Costs below are planning categories rather than verified live prices. Actual free-tier limits and provider pricing should be checked before implementation or deployment.

Service / Component	MVP Cost Strategy	Notes
React Native / Expo	Free / open-source tooling	Core app development
PostgreSQL	Local or free development tier	Production hosting is future scope
Node.js / Express	Free / open source	Backend runtime
WebRTC	Open source; infrastructure may vary	TURN/media infrastructure may incur cost
Groq API	Use available free/developer quota if sufficient	Quota and pricing must be verified at implementation time
EAS Build	Free/developer allowance where available	Limits depend on current plan
Push Notifications	Platform/service dependent	Use only required capabilities
Hosting	Free/developer tier for OJT if adequate	Avoid claiming zero cost for production

17. Project Roadmap



Week	Deliverable
W1	Scope, requirements, roles, architecture direction and stack
W2	Wireframes, screen designs and patient/doctor flows
W3	Navigation, database schema, state management and repository foundation
W4	Authentication, role-based dashboards and core navigation
W5	Doctor profiles, specialization, availability, slots and booking
W6	REST API integration, SQLite offline storage and synchronization
W7	WebRTC consultation, STT + AI draft summary, prescription workflow
W8	Doctor review/approval, integration, offline/error handling
W9	Jest tests, performance, dark mode, EAS build and stabilization
W10	Final integration, documentation, demo preparation and viva

17.1 Major Milestones

- M1: Authentication and role separation complete
- M2: Booking flow complete
- M3: Consultation + AI summary prototype complete
- M4: Doctor-approved record + prescription complete
- M5: Offline/sync and end-to-end testing complete
- M6: Final build and demonstration ready

18. Team Responsibilities

Workstream	Student 1	Student 2	Shared / Mentor
Authentication & Role Management	Implementation	Testing / integration	Mentor review
Patient UI	Primary	Support	UX review
Doctor UI	Support	Primary	UX review
Appointment / Slots	Primary	Support	Architecture review
Backend / API	Support	Primary	API review
SQLite / Offline Sync	Primary	Support	Sync review
WebRTC / Consultation	Support	Primary	Integration review
AI Summary / Prescription	Primary	Primary	Doctor-review workflow
Testing / Final Delivery	Shared	Shared	Mentor checkpoint

The exact division can be adjusted as implementation progresses; the important requirement is that both students understand the complete end-to-end workflow for the final viva.

19. GitHub Repository Structure

```
DoctorDirect/ ■■■■ mobile/ ■ ■■■■ src/ ■ ■■■■ assets/ ■ ■■■■ app.json ■ ■■■■ package.json ■■■■ backend/ ■  
■■■■ src/ ■ ■ ■■■■ controllers/ ■ ■ ■■■■ routes/ ■ ■ ■■■■ services/ ■ ■ ■■■■ middleware/ ■ ■ ■■■■  
models/ ■ ■ ■■■■ utils/ ■ ■■■■ package.json ■■■■ database/ ■ ■■■■ migrations/ ■ ■■■■ seed/ ■■■■ docs/ ■  
■■■■ BRD.md ■ ■■■■ PRD.md ■ ■■■■ UX.md ■ ■■■■ TRD.md ■ ■■■■ HLD.md ■ ■■■■ ADRs/ ■■■■ tests/ ■■■■ README.md
```

19.1 Repository Practices

- Use meaningful commit messages.
- Keep secrets out of Git; use environment variables.
- Document local setup and required API keys/services.
- Keep schema migrations versioned.
- Attach test evidence to major pull requests.

20. README

20.1 Project Description

DoctorDirect is an offline-first React Native healthcare application for patient-doctor appointment booking, tele-consultation, consultation records and digital prescriptions. It uses role-based workflows for patients and doctors, SQLite for selected offline data, a Node.js/Express backend with PostgreSQL, WebRTC for video consultation, and Groq API services for speech-to-text and AI-assisted draft summaries.

20.2 Suggested README Sections

- Features
- Technology Stack
- Architecture
- Prerequisites
- Environment Variables
- Installation
- Database Setup
- Run Mobile App
- Run Backend
- Testing
- Build / EAS
- Project Structure
- Limitations and Future Scope

20.3 Demo Flow

For the final presentation, demonstrate one continuous story: patient logs in → finds a doctor → books a slot → doctor sees appointment → both join consultation → temporary audio is processed → transcript becomes an AI draft → doctor reviews/approves → doctor creates/approves prescription → patient views the final record → offline access is demonstrated for synchronized records.

21. Architecture Decision Records (ADRs)

ADR	Decision	Rationale
ADR-001	React Native + Expo	Use one JavaScript/TypeScript mobile codebase for Android and iOS while simplifying native integration.
ADR-002	React Navigation	Use stack/tab navigation suitable for separate patient and doctor workflows.
ADR-003	SQLite for Offline Data	Keep selected synchronized healthcare records locally so they remain readable during connectivity loss.
ADR-004	PostgreSQL as Server Source of Truth	Use a relational server database for users, doctors, slots, appointments and medical-record metadata.
ADR-005	WebRTC for Consultation	Use real-time peer/media technology for patient-doctor video consultation.
ADR-006	Groq for STT + AI Draft	Use an online AI service to process temporary audio and generate a draft summary, subject to quota and availability.
ADR-007	Doctor Approval Before Final Record	AI output is never treated as a final medical record without doctor review and approval.
ADR-008	No Permanent Raw Video Storage in MVP	Avoid unnecessary retention of consultation video; process temporary audio for the summary workflow.
ADR-009	Server Authority for Appointment Conflicts	Offline state cannot guarantee slot exclusivity; final booking validity comes from the server.

Appendix — Scope Notes

- This document follows the structure and level of detail demonstrated by the provided FitTrack reference document, but the content is adapted to DoctorDirect.
- Where the current project material does not define an implementation detail, this document labels it as a proposed/recommended design rather than presenting it as an already implemented fact.
- Legal validity of electronic/digital signatures, healthcare regulatory compliance, and production deployment are outside the demonstrated OJT scope unless separately implemented and verified.

End of Document